

1. Factory Method Pattern

Where it's used:

In `views.py`, within the `intersection_detail` view when handling POST requests for adding vehicles.

How it's used:

When a user selects a vehicle type (Car, Bus, Truck) and submits the form, the view dynamically creates instances of the `Vehicle` model based on the selected type. This is a conceptual implementation of the Factory Method, where the creation logic for different vehicle types is encapsulated and abstracted.

Example Concept:

python

 Copy

 Edit

```
vehicle = Vehicle.objects.create(type='Car', approach=approach)
```

→ Here, different "products" (vehicle types) are instantiated based on user input.

2. Observer Pattern

Where it's used:

In emergency simulation within the `intersection_detail` view.

How it's used:

When an emergency is triggered for one approach (subject), all other approaches (observers) at the same intersection are notified and their states are changed to RED. This mimics the observer pattern, where a state change in one entity affects others.

Behavior:

python

 Copy

 Edit

```
if form.cleaned_data['trigger_emergency']:
    target_approach.current_state = 'GREEN'
    for other in all_other_approaches:
        other.current_state = 'RED'
```

3. State Pattern

Where it's used:

In the management of traffic light states via the `current_state` field in the `Approach` model.

How it's used:

Users can manually change an approach's state between Red, Yellow, and Green via the UI, simulating explicit transitions between predefined states. This closely resembles the classic State pattern, where an object's behavior changes based on its internal state.

Model Logic Example:

python

 Copy

 Edit

```
class Approach(models.Model):  
    current_state = models.CharField(choices=STATE_CHOICES)
```

4. Singleton Pattern

Where it's used:

In the architecture of the Django app itself.

How it's used:

Django's architecture inherently behaves like a Singleton by maintaining a single shared state across the app (managed by the database and request/response cycle). Your views control traffic states and emergency statuses for all intersections centrally, enforcing a single point of access and control.

In Practice:

- All intersections and approaches are accessed and updated through Django's ORM.
- A single set of view functions orchestrates all interactions with the traffic system.

Summary Table

Design Pattern	Role in Project
Factory Method	Creates vehicles dynamically from form input
Observer Pattern	Updates all approaches when one triggers an emergency
State Pattern	Manages transition between traffic light states
Singleton Pattern	Central controller via Django views + ORM as global manager